

# Terraform on AWS

*Environment Setup + Hands-On Lab: All Core .tf File Types*

Covers: Installing Terraform & AWS CLI on an EC2 Linux machine → IAM permissions → a complete lab using variables, locals, data sources, main resources, and outputs → validation.

# Part 1: Environment Setup — Terraform on an AWS Linux Machine

This section prepares a fresh EC2 Linux instance so it can run Terraform and authenticate to AWS.

## 1.1 Launch the EC2 Machine

- EC2 Console → Launch Instance
- AMI: Amazon Linux 2023 (or Ubuntu 22.04 — commands below cover both)
- Instance type: t2.micro / t3.micro
- Security group: allow inbound SSH (port 22)
- Attach an IAM role to this instance with EC2 permissions (see 1.4) — this avoids storing access keys on the box
- Launch, then connect:

```
ssh -i your-key.pem ec2-user@<public-ip>      # Amazon Linux
ssh -i your-key.pem ubuntu@<public-ip>         # Ubuntu
```

## 1.2 Install Terraform

1. Update the system:

```
sudo yum update -y          # Amazon Linux
# --- OR ---
sudo apt update -y          # Ubuntu
```

2. Add HashiCorp's repo and install:

```
# Amazon Linux / RHEL
sudo yum install -y yum-utils
sudo yum-config-manager --add-repo
https://rpm.releases.hashicorp.com/AmazonLinux/hashicorp.repo
sudo yum install -y terraform

# --- Ubuntu / Debian ---
sudo apt install -y gnupg software-properties-common curl
wget -O- https://apt.releases.hashicorp.com/gpg | sudo gpg --dearmor -o
/usr/share/keyrings/hashicorp-archive-keyring.gpg
echo "deb [signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg]
https://apt.releases.hashicorp.com $(lsb_release -cs) main" | sudo tee
/etc/apt/sources.list.d/hashicorp.list
sudo apt update && sudo apt install -y terraform
```

3. Verify:

```
terraform -version
```

### ✓ Validation Steps

4. Command prints a Terraform version (e.g. Terraform v1.9.x) with no errors.

## 1.3 Install & Configure the AWS CLI

```
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"
sudo yum install -y unzip      # or: sudo apt install -y unzip
unzip awscliv2.zip
sudo ./aws/install
aws --version
```

If the EC2 instance already has an IAM role attached (recommended), you do NOT need to run “aws configure” — credentials are provided automatically via the instance metadata service.

If you are not using an instance role, configure access keys manually instead:

```
aws configure
AWS Access Key ID: <your-key>
```

```
AWS Secret Access Key: <your-secret>
Default region name: ap-south-1
Default output format: json
```

## ✓ Validation Steps

5. ``aws sts get-caller-identity`` returns your Account ID and the role/user ARN with no errors.

## 1.4 IAM Permissions the Role/User Needs

Attach at least EC2 permissions to whichever IAM role or user Terraform will run as. For a lab environment, the quickest option is the AWS managed policy:

- IAM → Roles → select your role → Add permissions → Attach policies → search “AmazonEC2FullAccess” → Add

For least-privilege, use a scoped inline policy instead:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "ec2:RunInstances",
        "ec2:DescribeInstances",
        "ec2:TerminateInstances",
        "ec2:CreateTags",
        "ec2:DescribeImages",
        "ec2:DescribeInstanceTypes",
        "ec2:DescribeSecurityGroups",
        "ec2:DescribeSubnets",
        "ec2:DescribeVpcs",
        "ec2:DescribeKeyPairs"
      ],
      "Resource": "*"
    }
  ]
}
```

**Common error:** “You are not authorized to perform: ec2:RunInstances” means the IAM role attached to your instance is missing EC2 permissions — attach the policy above and retry within a minute or two.

## Part 2: Terraform File Types — Quick Reference

Before the lab, here is a quick reference of every file type used in it.

- `variables.tf` — Declares input variables (name + optional default/type)
- `terraform.tfvars` — Supplies actual values for those variables; auto-loaded by Terraform
- `locals.tf` — Computed/derived values used internally, not passed in externally
- `data.tf` — Read-only lookups against existing cloud resources (e.g. latest AMI)
- `main.tf` — The actual resource blocks — what gets created
- `outputs.tf` — Values to print after apply (IDs, IPs, ARNs)
- `versions.tf` — Pins Terraform and provider versions for consistency
- `terraform.tfstate` — Auto-generated; Terraform's record of what currently exists

## Part 3: Hands-On Lab — All File Types Together

Objective: build one small AWS project using variables.tf, terraform.tfvars, locals.tf, data.tf, main.tf, outputs.tf, and versions.tf, and confirm each file's role.

### Step 1: Create the Project Folder and Files

```
mkdir ~/terraform-lecture-lab && cd ~/terraform-lecture-lab
touch variables.tf terraform.tfvars locals.tf data.tf main.tf outputs.tf versions.tf
```

### Step 2: versions.tf

```
terraform {
  required_providers {
    aws = {
      source = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}
```

### Step 3: variables.tf

```
variable "region" {
  default = "ap-south-1"
}

variable "instance_type" {
  default = "t2.micro"
}
```

### Step 4: terraform.tfvars

```
region      = "ap-south-1"
instance_type = "t2.micro"
```

### Step 5: locals.tf

```
locals {
  common_tags = {
    Name      = "Lecture-Lab-Instance"
    ManagedBy = "Terraform"
  }
}
```

### Step 6: data.tf

```
data "aws_ami" "amazon_linux" {
  most_recent = true
  owners      = ["amazon"]

  filter {
    name   = "name"
    values = ["al2023-ami-*-x86_64"]
  }
}
```

### Step 7: main.tf

```
provider "aws" {
  region = var.region
}
```

```
resource "aws_instance" "lab" {
  ami           = data.aws_ami.amazon_linux.id
  instance_type = var.instance_type
  tags          = local.common_tags
}
```

## Step 8: outputs.tf

```
output "instance_id" {
  value = aws_instance.lab.id
}

output "public_ip" {
  value = aws_instance.lab.public_ip
}

output "ami_used" {
  value = data.aws_ami.amazon_linux.id
}
```

## Step 9: Run the Lab

```
terraform init
terraform plan
terraform apply -auto-approve
```

### ✓ Validation Steps

6. `terraform output` shows instance\_id, public\_ip, and ami\_used with real values.
7. `ami\_used` prints a value starting with ami- — confirms the data source resolved a real, current AMI rather than a hardcoded one.
8. AWS Console → EC2 → Instances shows the instance tagged “Lecture-Lab-Instance” and running.
9. `terraform state list` shows both aws\_instance.lab and data.aws\_ami.amazon\_linux tracked.
10. `terraform destroy -auto-approve` removes the instance; confirm it disappears from the EC2 console.

## Part 4: Detailed Explanation of Every File in the Lab

This section explains what each file does, why it exists as a separate file, and what happens if you get it wrong.

### versions.tf

Purpose: tells Terraform which version of itself, and which version of the AWS provider plugin, this project was written for.

- `required_version` ensures nobody accidentally runs this project with an incompatible (too old/new) Terraform binary.
- `required_providers` pins the aws provider to a version range (`~> 5.0` means “5.x, but not 6.0”) so a provider update elsewhere on the system doesn't silently break this project.

**Why it matters:** Without version pinning, a teammate running a newer Terraform or provider version might get subtly different behavior, or the project might fail to init at all. This file is what makes a project reproducible across machines and over time.

### variables.tf

Purpose: declares the named inputs this configuration accepts, without assigning them a specific value yet.

- Each variable block is a placeholder — `region` and `instance_type` in our lab.
- The default argument is optional. If present, Terraform uses it when no other value is supplied. If absent, Terraform will prompt interactively (or fail in non-interactive/CI runs).
- You can also add a type (string, number, bool, list, map) so Terraform validates the input before it ever reaches a resource.

**Why it matters:** Variables are what make a Terraform project reusable. Without them, you'd need a separate copy of `main.tf` for every environment (dev/staging/prod) instead of one file reused with different values.

### terraform.tfvars

Purpose: supplies the actual values for the variables declared above.

- Terraform automatically loads a file literally named `terraform.tfvars` (or any file ending in `.auto.tfvars`) with no extra flag needed.
- Custom-named files, e.g. `dev.tfvars`, require an explicit flag: `terraform apply -var-file="dev.tfvars"`.

**Why it matters:** This is the file you change per environment or per student, while `variables.tf` and `main.tf` stay identical. It's also the file most teams keep OUT of shared Git repos if it contains anything sensitive (like a password), using a `.gitignore` entry.

### locals.tf

Purpose: defines values Terraform computes/derives internally, as opposed to values supplied externally (which is what variables are for).

- In the lab, `common_tags` is a local — a small map of tags reused across multiple resources so they all stay consistent.
- Locals can also reference variables or other locals, e.g. combining `var.region` with a naming convention to build a resource name automatically.

**Why it matters:** Locals avoid repeating the same expression in ten different places. If the tagging convention changes later, you edit it once in `locals.tf` instead of hunting through every resource block.

### data.tf

Purpose: performs a read-only lookup against something that already exists in the cloud, instead of hardcoding a value.

- The `aws_ami` data source in the lab asks AWS: “what is the most recent Amazon Linux 2023 AMI right now?” and returns its ID.
- Unlike a resource block, a data block never creates, changes, or destroys anything — it only reads.

**Why it matters:** AMI IDs change every time AWS releases a new image, and they also differ per AWS region. A data source keeps the lab working correctly forever, instead of silently launching an outdated or region-mismatched image the way a hardcoded AMI ID would eventually do.

## main.tf

Purpose: the actual resource block(s) — the real infrastructure Terraform will create.

- The provider block tells Terraform which cloud and region to talk to.
- The resource block defines one EC2 instance, referencing the AMI from data.tf, the instance type from var.instance\_type, and the tags from local.common\_tags.

**Why it matters:** This is the only file that actually causes something to be created. Everything else (variables, locals, data) exists purely to feed clean, reusable values into this file.

## outputs.tf

Purpose: prints useful values after apply finishes, and makes them available to other tools or Terraform configurations.

- instance\_id and public\_ip are attributes of the resource we just created — not known until AWS actually creates the instance (hence “known after apply” during plan).
- ami\_used echoes back the AMI ID the data source resolved, which is a good way to prove to students the lookup actually worked.

**Why it matters:** Outputs are also how one Terraform project can hand a value to another (e.g. a VPC ID from a networking project passed into an application project), and how CI/CD pipelines grab values like a load balancer DNS name after a deploy.



## Part 5: The State File and Other Auto-Generated Files

Beyond the files you write, Terraform creates several files and folders of its own the first time you run `init`, `plan`, or `apply`. This section explains each one in detail.

### **terraform.tfstate — the most important generated file**

This is a JSON file that is Terraform's memory of exactly what it has created: every resource's ID, every attribute AWS returned, and how resources relate to each other.

- Created the moment you run your first terraform `apply`.
- Read by terraform `plan` every time, to compare “what the `.tf` files say should exist” against “what state says already exists.”
- Updated by terraform `apply` and terraform `destroy` after every change.
- Contains real data returned by AWS — including, in some cases, sensitive values — so it should never be committed to a public Git repository in plain form.

**What goes wrong if it's lost or edited by hand:** If `terraform.tfstate` is deleted, Terraform “forgets” the EC2 instance it created. The next `plan` will think nothing exists yet and try to create a duplicate instance — even though the original is still running and now costing money untracked. If the file is hand-edited incorrectly, Terraform may believe a resource has attributes it doesn't actually have, causing confusing `plan` output or failed `applies`.

### **terraform.tfstate.backup**

An automatic backup containing the state exactly as it was immediately before the most recent write.

- Created/overwritten automatically every time `terraform.tfstate` is about to be updated.
- Acts as a one-step-back safety net — if an `apply` is interrupted or corrupts the state file, this backup can sometimes be used to manually recover.
- Only keeps one previous version, not a full history — it is not a substitute for proper version-controlled remote state in a team setting.

### **.terraform/ (folder)**

Created by terraform `init`. Contains the downloaded provider plugin binaries (the actual aws provider code Terraform Core calls to talk to AWS).

- Can be safely deleted at any time — running terraform `init` again will re-download everything.
- Usually excluded from Git via `.gitignore`, since it's large and machine-specific.

### **.terraform.lock.hcl**

Also created by `init`. Records the exact version and cryptographic checksum of every provider that was downloaded.

- Ensures that everyone on a team — and every CI/CD run — downloads the identical provider version and binary, not just “some version matching `>= 5.0`.”
- Unlike `.terraform/`, this file **SHOULD** be committed to version control, so the exact provider version is locked for everyone.

## **Remote state (why labs move state off the local machine)**

By default, `terraform.tfstate` lives as a plain file on whichever machine ran `apply`. That's fine for a solo lab, but breaks down the moment more than one person or machine needs to manage the same infrastructure.

- Remote state stores `terraform.tfstate` in a shared location instead — commonly an S3 bucket (AWS) or a Storage Account (Azure).
- A state lock (commonly a DynamoDB table on AWS) prevents two people from running `apply` at the exact same time and corrupting the state file.

```
terraform {  
  backend "s3" {  
    bucket      = "my-terraform-state-bucket"  
    key         = "lecture/terraform.tfstate"  
    region      = "ap-south-1"  
    dynamodb_table = "terraform-lock"  
  }  
}
```

**Teaching takeaway:** Local state (the default) is fine for this lecture's lab. Remote state becomes necessary the moment a second person, a CI/CD pipeline, or a second laptop needs to manage the same set of resources.

### ✓ Validation Steps

11. Run `ls -la` in the lab folder after apply — confirm `terraform.tfstate`, `.terraform.lock.hcl`, and the `.terraform/` folder all now exist.
12. Open `terraform.tfstate` in a text editor (read-only, do not save/edit) and locate the `aws_instance.lab` resource block — confirm its `instance_id` and `ami` attributes match what's shown in the AWS Console.
13. Run `terraform apply` a second time with no changes — confirm it now creates a `terraform.tfstate.backup` file and reports “No changes.”
14. Delete `.terraform/` only (not the state file) and re-run `terraform init` — confirm it re-downloads the provider and everything still works.

*End of Guide*